Análise Minuciosa do Projeto Workflow-Python

Mapeamento estrutural, diagramas de classes, atividades e sequências

Projeto analisado	Workflow-Python-main (Django + Django REST Framework)
Artefatos inspecionados	README, settings, urls, models, serializers, views, migrations, banco SQLite e DOC/usecase.mmd
Escopo desta entrega	Relatório analítico + diagramas consolidados em PDF

Gerado em 29/03/2026 15:11.

1. Visão executiva

O projeto analisado é uma API em Django/DRF voltada para gerenciamento de usuários, habilidades e ordens de serviço. A arquitetura funcional real é menor do que o repositório sugere: no estado atual, o fluxo ativo concentra-se nos apps `usuarios` e `ordens`, enquanto o app `freelancers` permanece como código legado ou incompleto.

- Padrão arquitetural predominante: Model + Serializer + ViewSet com roteamento por DefaultRouter.
- Modelo central de negócio: Usuario, Habilidade e OrdemDeServico.
- Papel de freelancer no fluxo ativo: atributo booleano do Usuario; não há entidade operacional separada.
- Persistência: SQLite com migrações ativas para usuarios e ordens.
- Autenticação: apenas SessionAuthentication do DRF; login/JWT não foram implementados como API de negócio.

2. Metodologia de análise

A análise combinou leitura estática do código-fonte, inspeção estrutural do banco SQLite incluído no projeto e comparação entre o comportamento prometido no README e o comportamento realmente exposto nas configurações e rotas. Os diagramas foram derivados diretamente das classes, relações ORM, serializers, viewsets e casos de uso declarados no arquivo DOC/usecase.mmd.

3. Estrutura geral do projeto

Área	Papel	Observação
workflow/	Configuração do projeto	Contém settings.py, urls.py, asgi.py e wsgi.py.
usuarios/	Domínio ativo	Modela Habilidade e Usuario, além de serializers e viewsets.
ordens/	Domínio ativo	Modela OrdemDeServico e seu serializer/viewset.
freelancers/	Código latente/legado	Existe no repositório, mas não está habilitado em INSTALLED_APPS nem em workflow/urls.py.
DOC/usecase.mmd	Visão de requisitos	Declara 10 casos de uso do sistema, alguns só parcialmente materializados no código.
db.sqlite3	Banco de desenvolvimento	Confirma a presença apenas das tabelas ativas do fluxo usuarios + ordens.

Instantâneo do banco incluído no projeto:

Tabela	Quantidade de registros
tab_habilidade	9
tab_usuario	1
tab_ordem_servico	3
tab_usuario_habilidades	1
tab_ordem_servico_habilidades_necessarias	12
tab_ordem_servico_freelancers_candidatos	0

4. Componentes que estão realmente operacionais

- Ativos pelas configurações: usuarios e ordens (workflow/settings.py).
- Rotas registradas: /api/usuarios/, /api/habilidades/, /api/ordens/ e /api-auth/ (workflow/urls.py).
- O README cita /api/freelancers/, mas a rota não aparece no DefaultRouter.
- As tabelas do banco confirmam ausência do modelo Freelancer em uso real.

5. Diagrama(s) de classes extraídos do projeto

Abaixo estão os diagramas de classes derivados do código. Para atender ao pedido de “todos os possíveis diagramas de classes”, o relatório separa: (a) o domínio consolidado, (b) o módulo de usuários, (c) o módulo de ordens e (d) o módulo freelancers como estrutura latente.

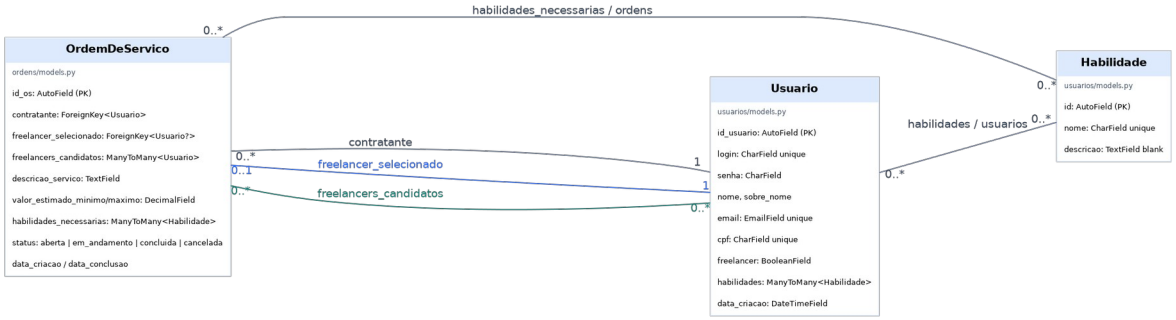

Figura 1 — Diagrama de classes consolidado do domínio ativo.

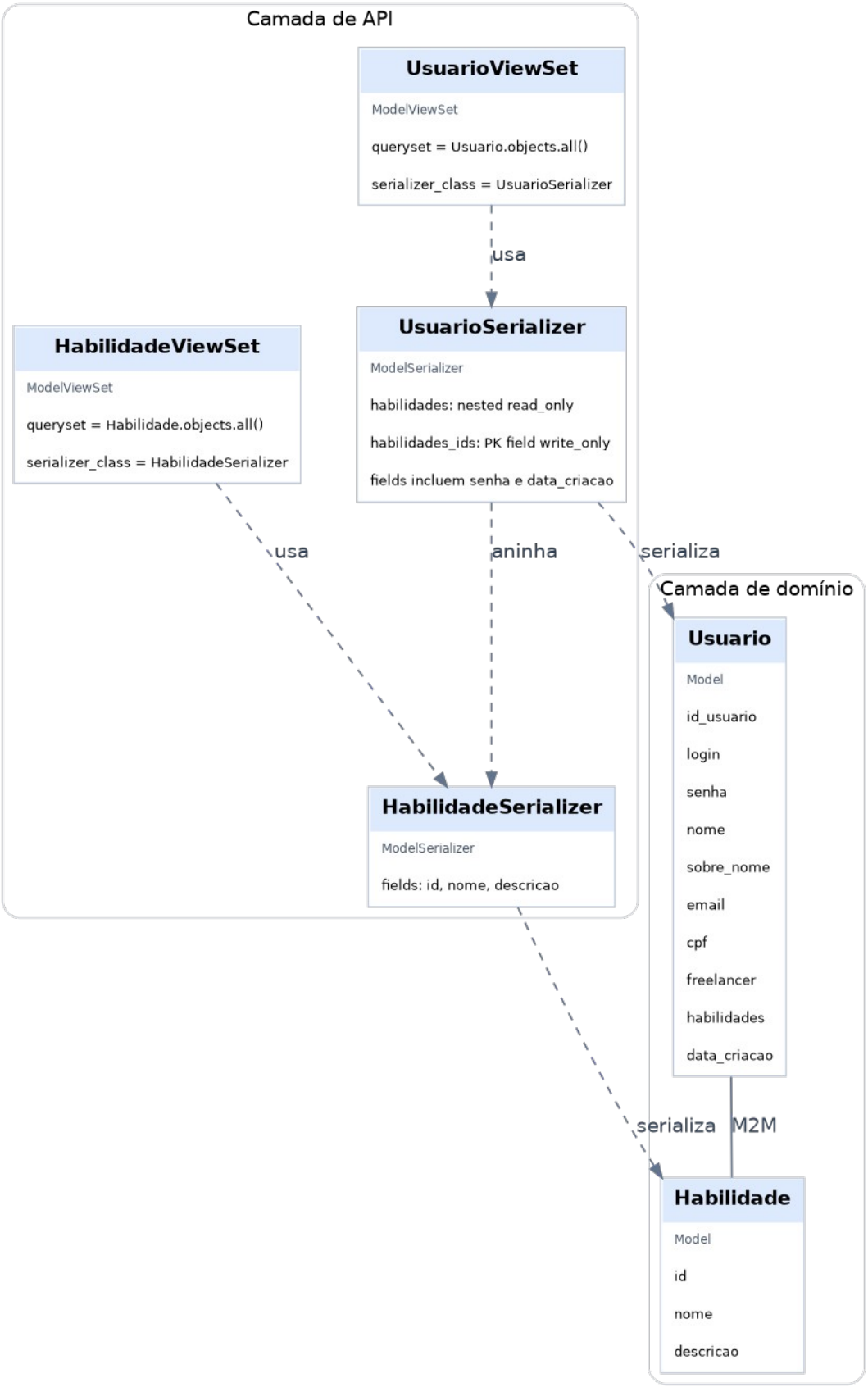
Camada de API
UsuarioViewSet
ModelViewSet
queryset = Usuario.objects.all()
serializer_class = UsuarioSerializer
usa
UsuarioSerializer
ModelSerializer
habilidades: nested read_only
habilidades_ids: PK field write_only
fields incluem senha e data_criacao
HabilidadeViewSet
ModelViewSet
queryset = Habilidade.objects.all()
serializer_class = HabilidadeSerializer
usa
aninha
serializa
Camada de domínio
Usuario
Model
id_usuario
login
senha
nome
sobre_nome
email
cpf
freelancer
habilidades
data_criacao
HabilidadeSerializer
ModelSerializer
fields: id, nome, descricao
serializa
M2M
Habilidade
Model
id
nome
descricao

Figura 2 — Diagrama de classes do módulo usuarios.

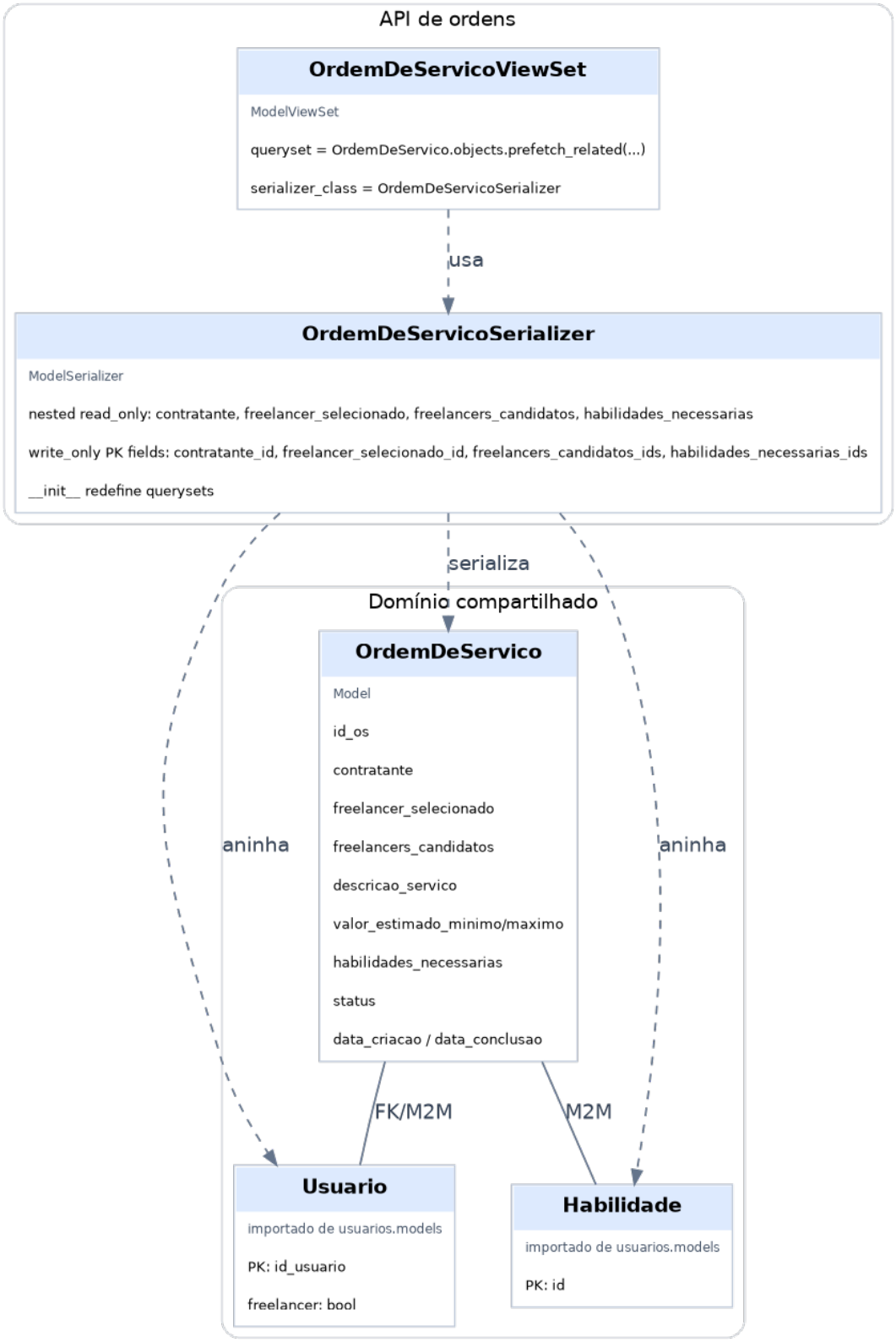

Figura 3 — Diagrama de classes do módulo ordens.

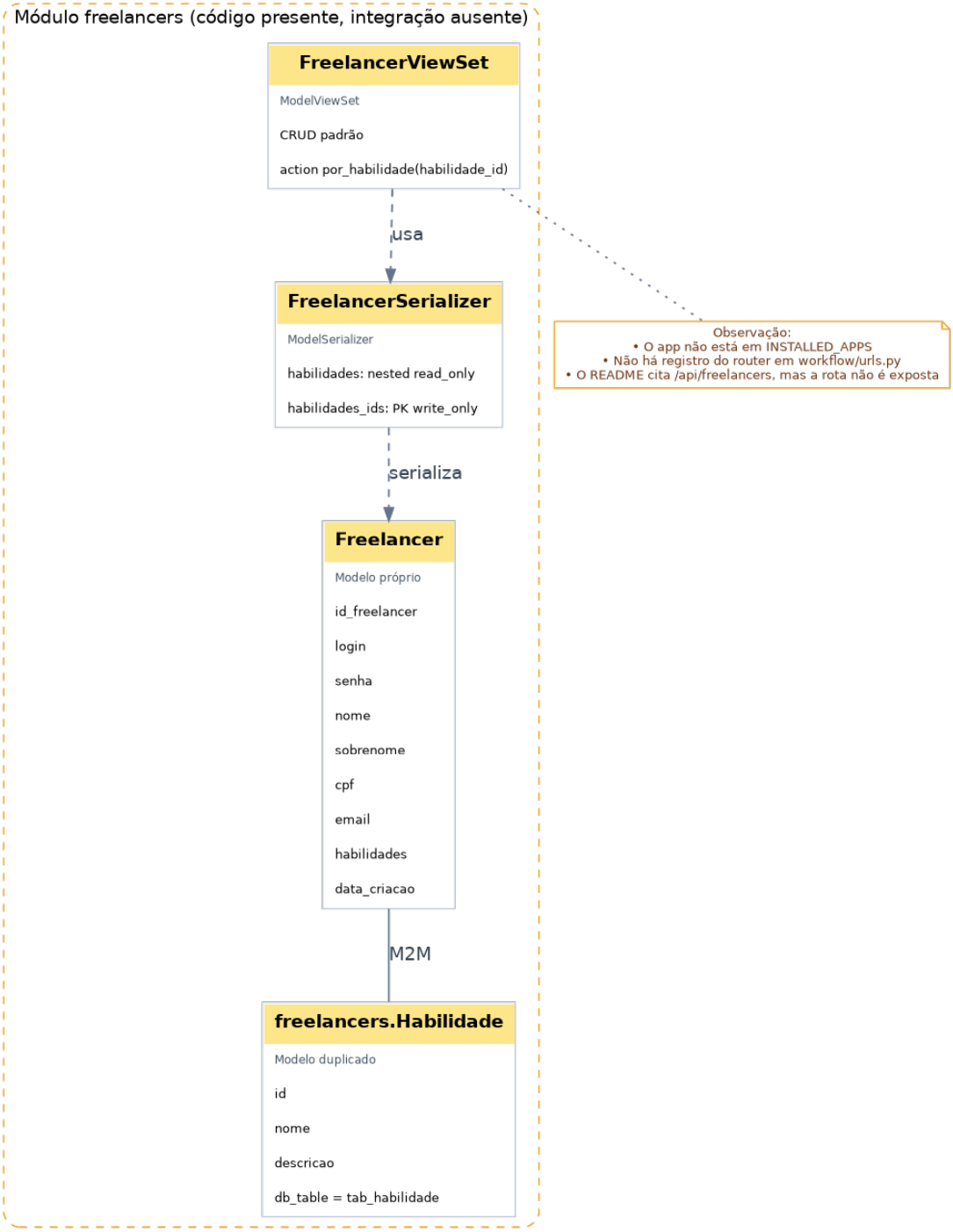

Figura 4 — Diagrama de classes do módulo freelancers, classificado como legado/latente.

6. Inventário analítico das classes

Classe	Camada	App	Responsabilidade	Situação
Habilidade	Model	usuarios	Catálogo de competências técnicas	Ativa
Usuario	Model	usuarios	Representa	Ativa

			pessoa/conta e também o papel de freelancer	
HabilidadeSerializer	Serializer	usuarios	Serializa Habilidade	Ativa
UsuarioSerializer	Serializer	usuarios	Serializa Usuario e relações com habilidades	Ativa
HabilidadeViewSet	ViewSet	usuarios	CRUD de Habilidade	Ativa
UsuarioViewSet	ViewSet	usuarios	CRUD de Usuario	Ativa
OrdemDeServico	Model	ordens	Representa a ordem e seu ciclo de vida	Ativa
OrdemDeServicoSerializer	Serializer	ordens	Serializa a ordem com relações nested	Ativa
OrdemDeServicoViewSet	ViewSet	ordens	CRUD da ordem	Ativa
freelancers.Habilidade	Model	freelancers	Duplica conceito de habilidade	Latente
Freelancer	Model	freelancers	Entidade separada para profissional	Latente
FreelancerSerializer	Serializer	freelancers	Serializa Freelancer	Latente
FreelancerViewSet	ViewSet	freelancers	CRUD + filtro por habilidade	Latente

7. Rotas e operações expostas

Rota base	Operações padrão	Origem	Observação
/api/usuarios/	GET, POST, GET{id}, PUT, PATCH, DELETE	DefaultRouter + UsuarioViewSet	CRUD completo ativo.
/api/habilidades/	GET, POST, GET{id}, PUT, PATCH, DELETE	DefaultRouter + HabilidadeViewSet	CRUD completo ativo.
/api/ordens/	GET, POST, GET{id}, PUT, PATCH, DELETE	DefaultRouter + OrdemDeServicoViewSet	CRUD completo ativo.
/api-auth/	login/logout da interface DRF	include(rest_framework.urls)	Suporte à SessionAuthentication.
/api/freelancers/	seria CRUD + action por_habilidade	README / app freelancers	Não está registrado nas rotas reais.

8. Casos de uso e atividades possíveis

O arquivo DOC/usecase.mmd lista 10 casos de uso. Nem todos aparecem como ações explícitas na API, mas todos podem ser classificados como implementados, parcialmente implementados ou apenas previstos. Em seguida, os diagramas de atividades representam tanto o que já está funcional quanto o que é inferido da estrutura atual.

Caso de uso	Fonte	Status	Leitura analítica
Registrar-se	DOC/usecase.mmd + UsuarioViewSet	**Implementado**	POST /api/usuarios/ cria usuários.
Fazer Login	DOC/usecase.mmd	**Parcial/indireto**	Há apenas /api-auth/ com SessionAuthentication; não há endpoint de login de negócio.
Atualizar Perfil	DOC/usecase.mmd + UsuarioViewSet	**Implementado**	PUT/PATCH em /api/usuarios/{id}/.
Tornar-se Freelancer	DOC/usecase.mmd + Usuario.freelancer	**Implementado indiretamente**	O papel é definido pelo booleano freelancer.

Criar Ordem	DOC/usecase.mmd + OrdemDeServicoViewSet	**Implementado**	POST /api/ordens/.
Listar Ordens	DOC/usecase.mmd + OrdemDeServicoViewSet	**Implementado**	GET /api/ordens/.
Candidatar-se a Ordem	DOC/usecase.mmd + freelancers_candidatos	**Parcial**	Inferido pela atualização do ManyToMany freelancers_candidatos.
Selecionar Freelancer	DOC/usecase.mmd + freelancer_selecionado	**Parcial**	Inferido por PATCH/PUT da ordem.
Executar Serviço	DOC/usecase.mmd + status	**Parcial**	Representado apenas pela mudança do status para em_andamento.
Concluir Ordem	DOC/usecase.mmd + status/data_conclusao	**Parcial**	Representado por status=concluida e data_conclusao.

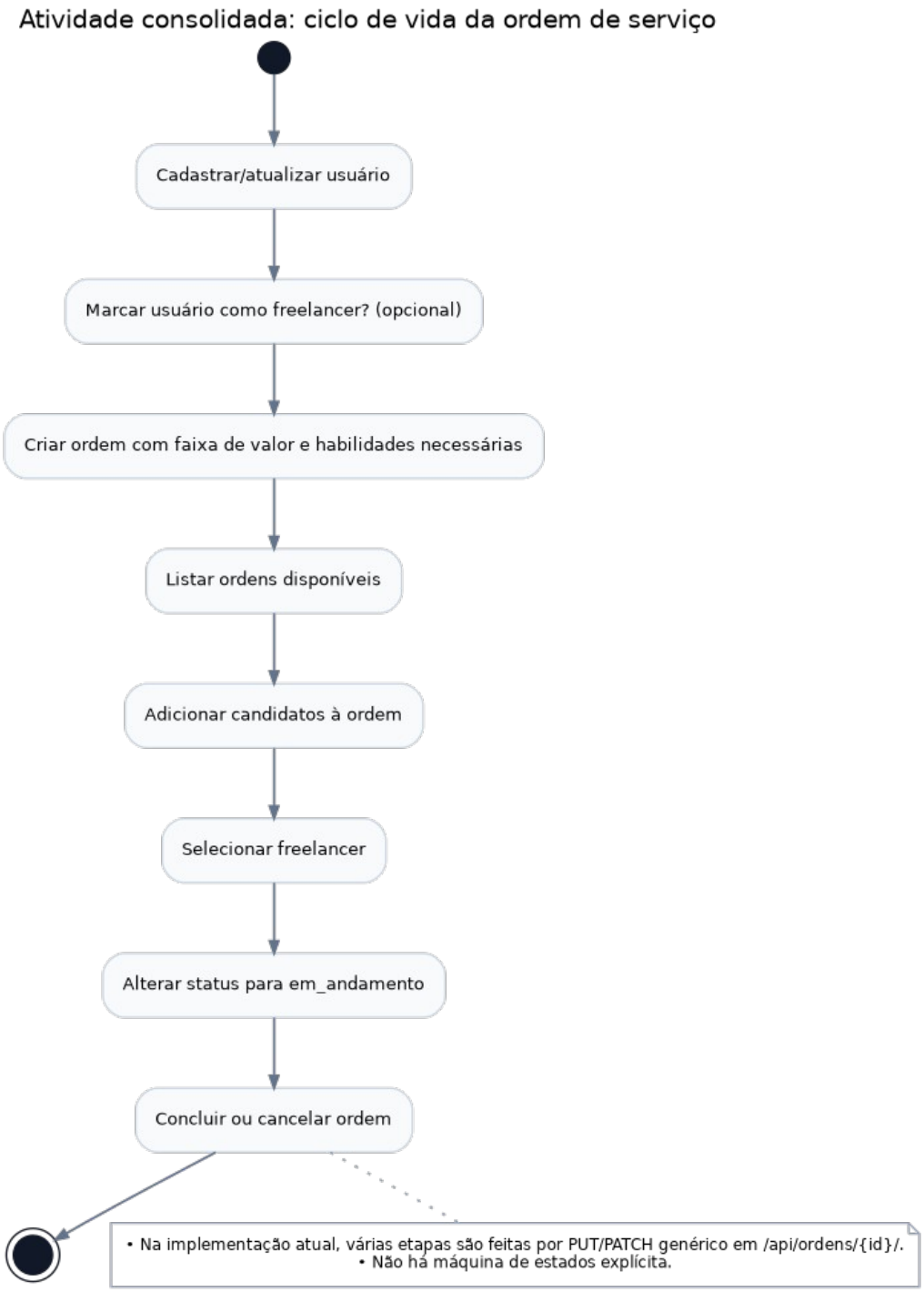

Figura 5 — Atividade consolidada do ciclo de vida da ordem.

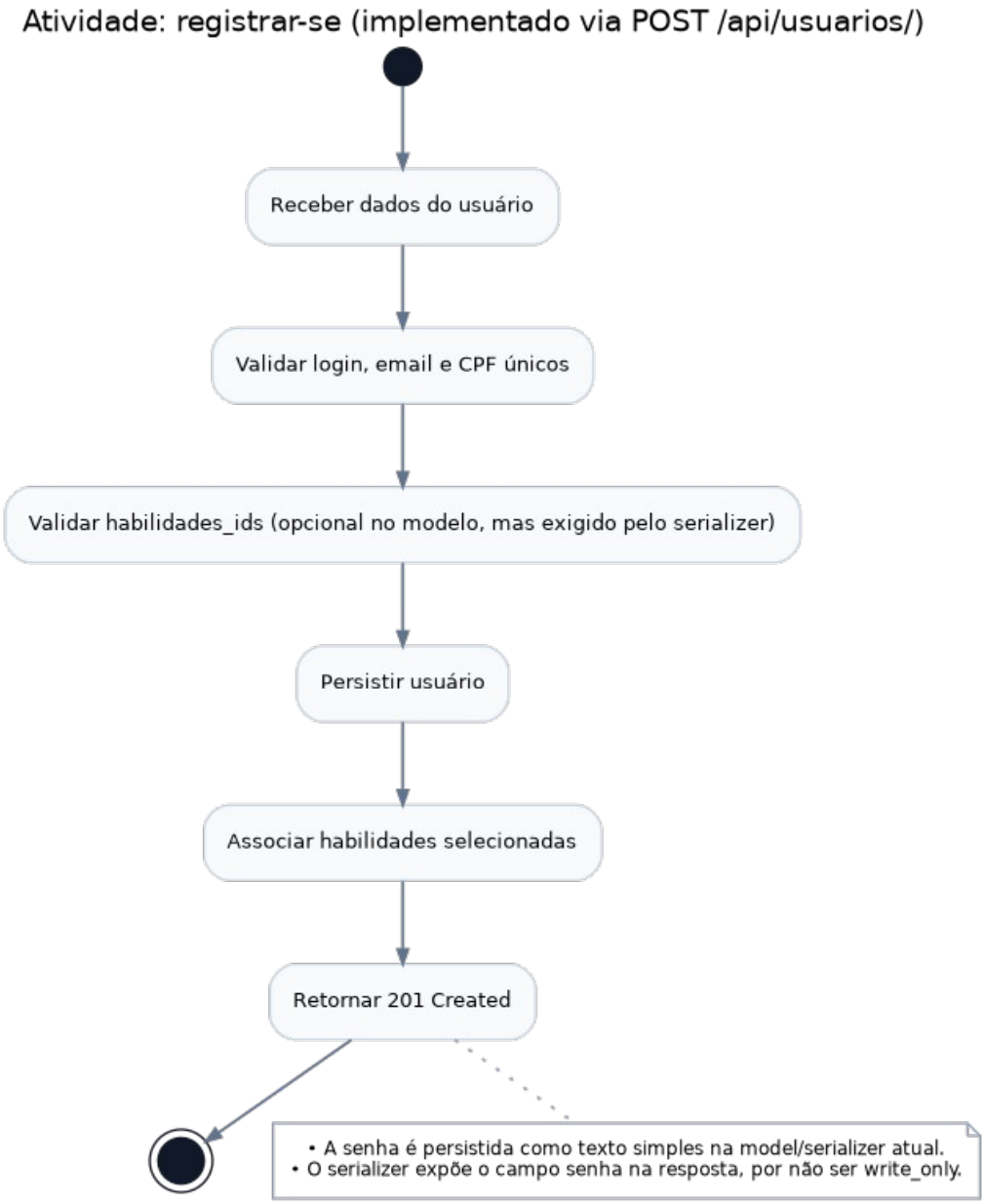

Figura 6 — Atividade de registro de usuário.

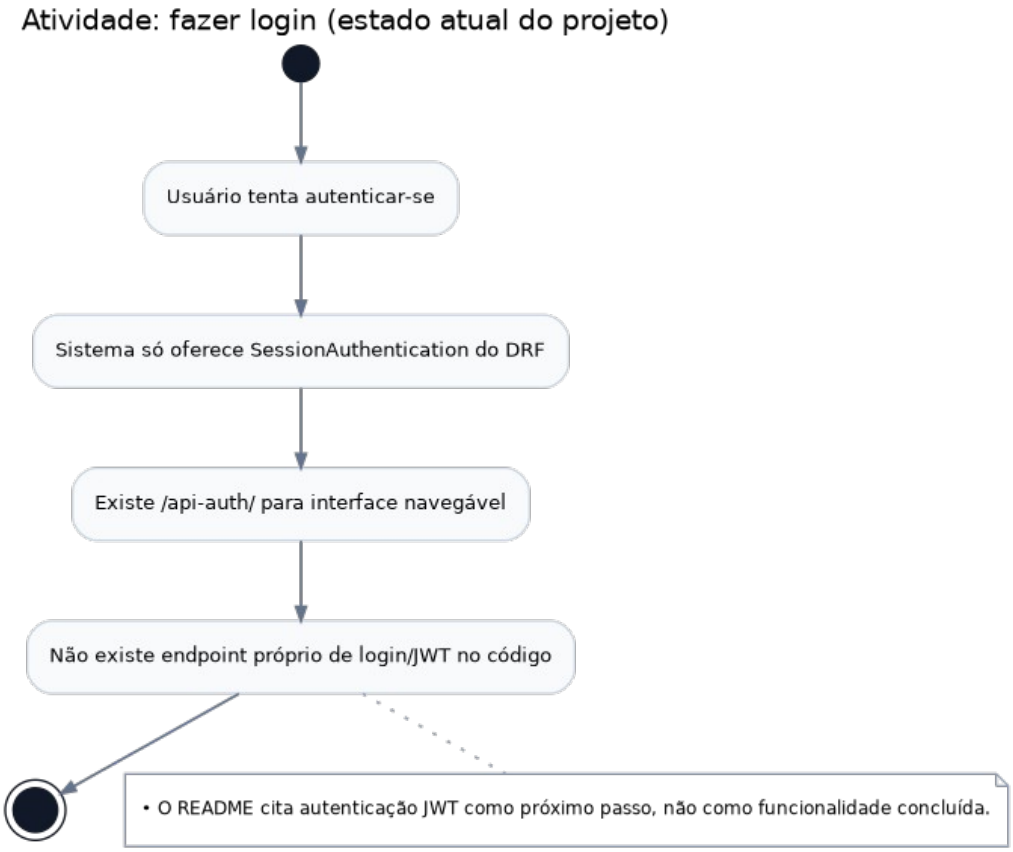

Figura 7 — Atividade de login no estado atual do projeto.

Atividade: atualizar perfil do usuário

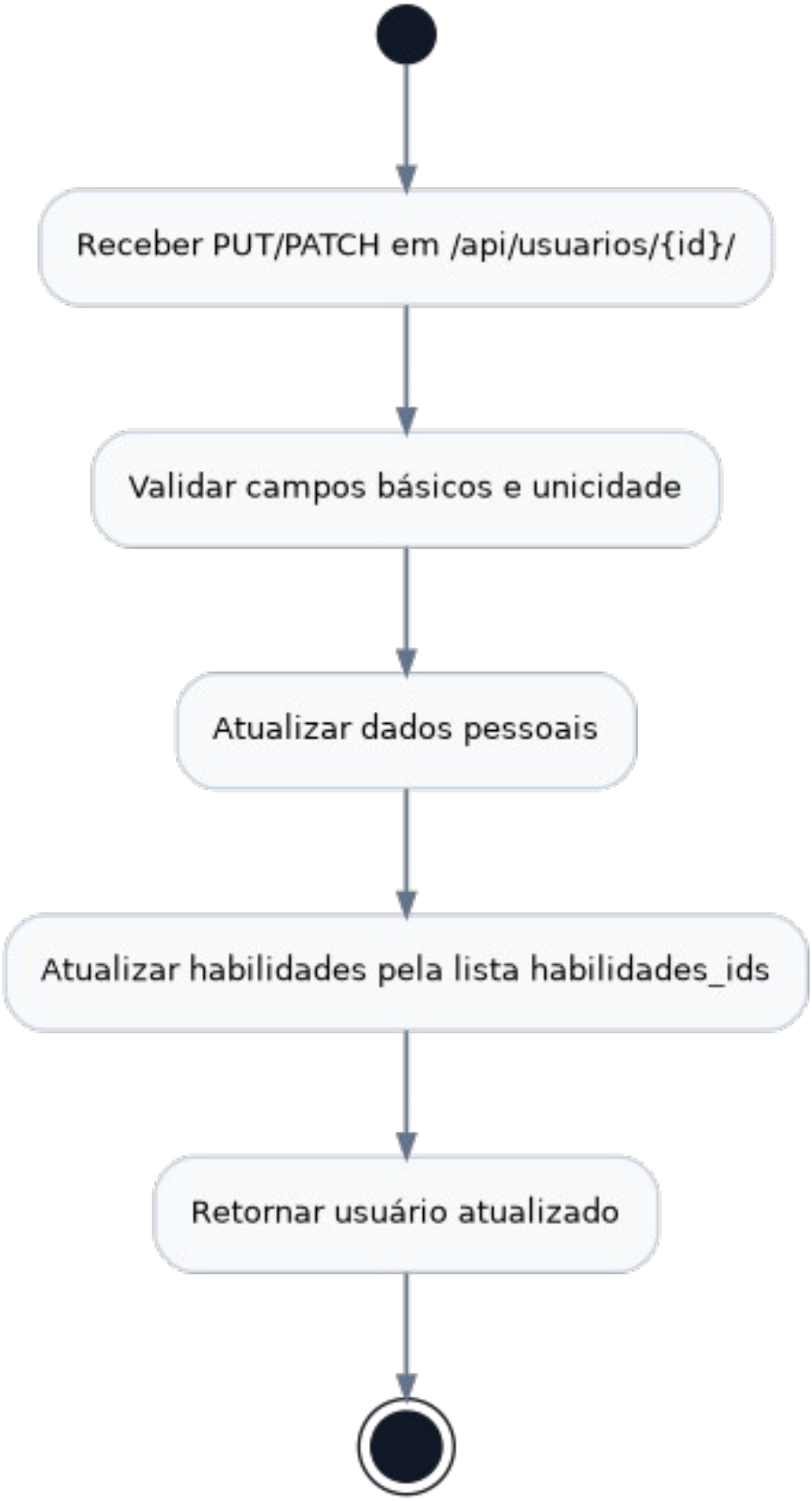

Figura 8 — Atividade de atualização de perfil.

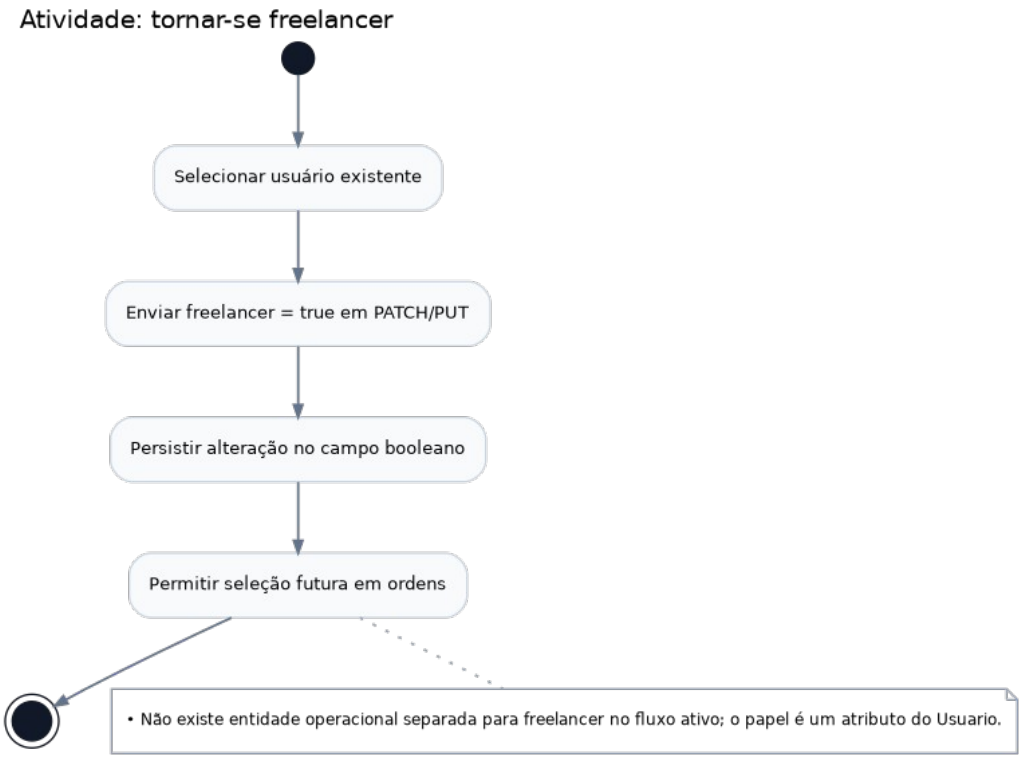

Figura 9 — Atividade para tornar-se freelancer.

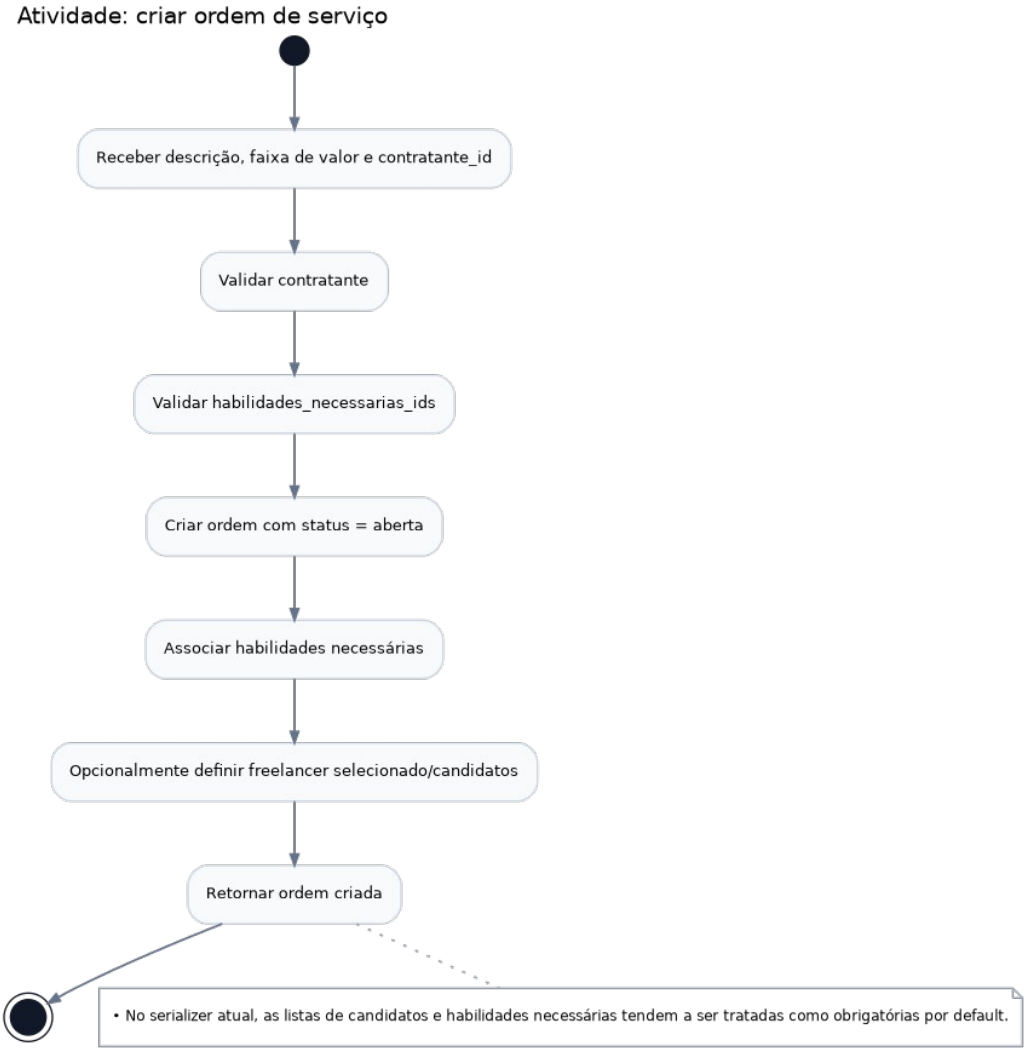

Figura 10 — Atividade de criação da ordem de serviço.

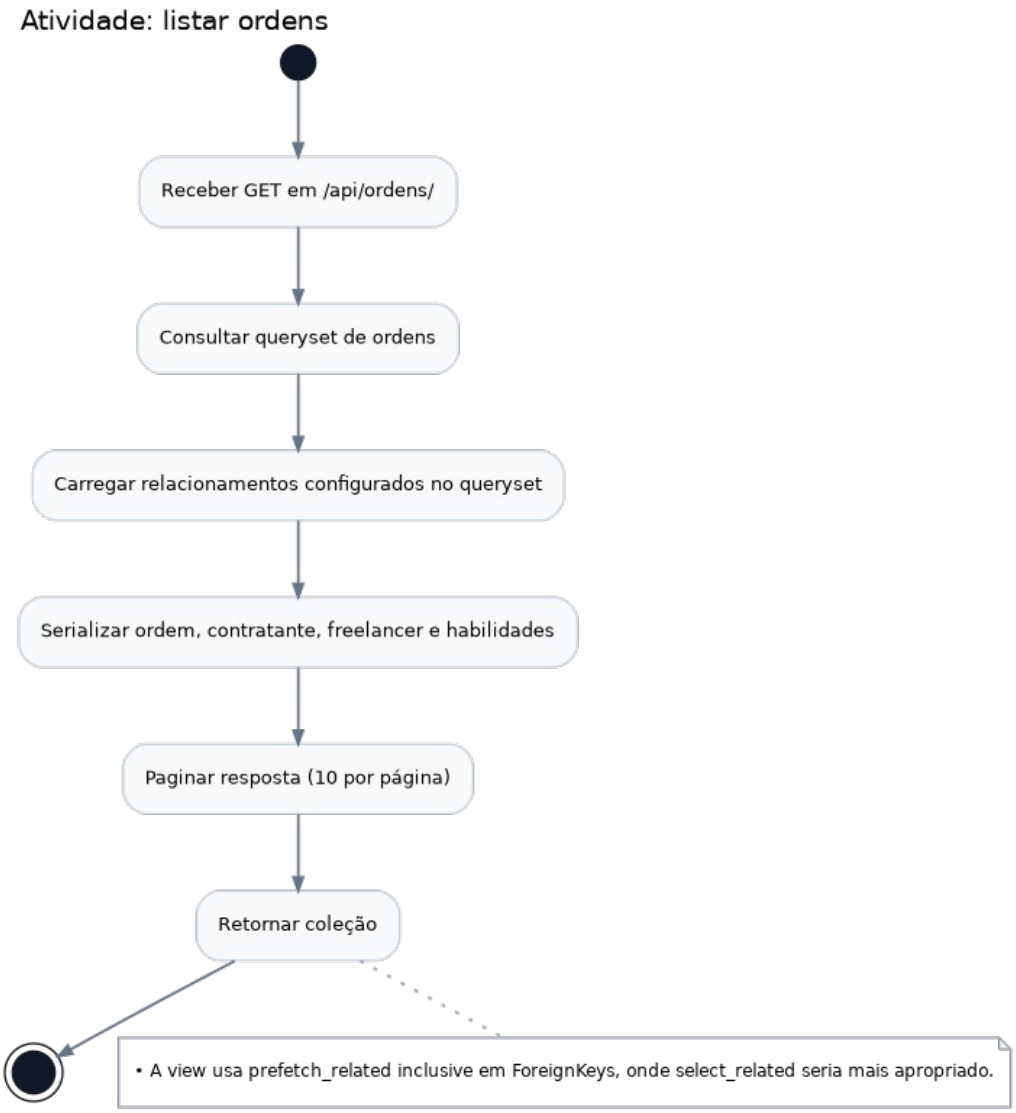

Figura 11 — Atividade de listagem de ordens.

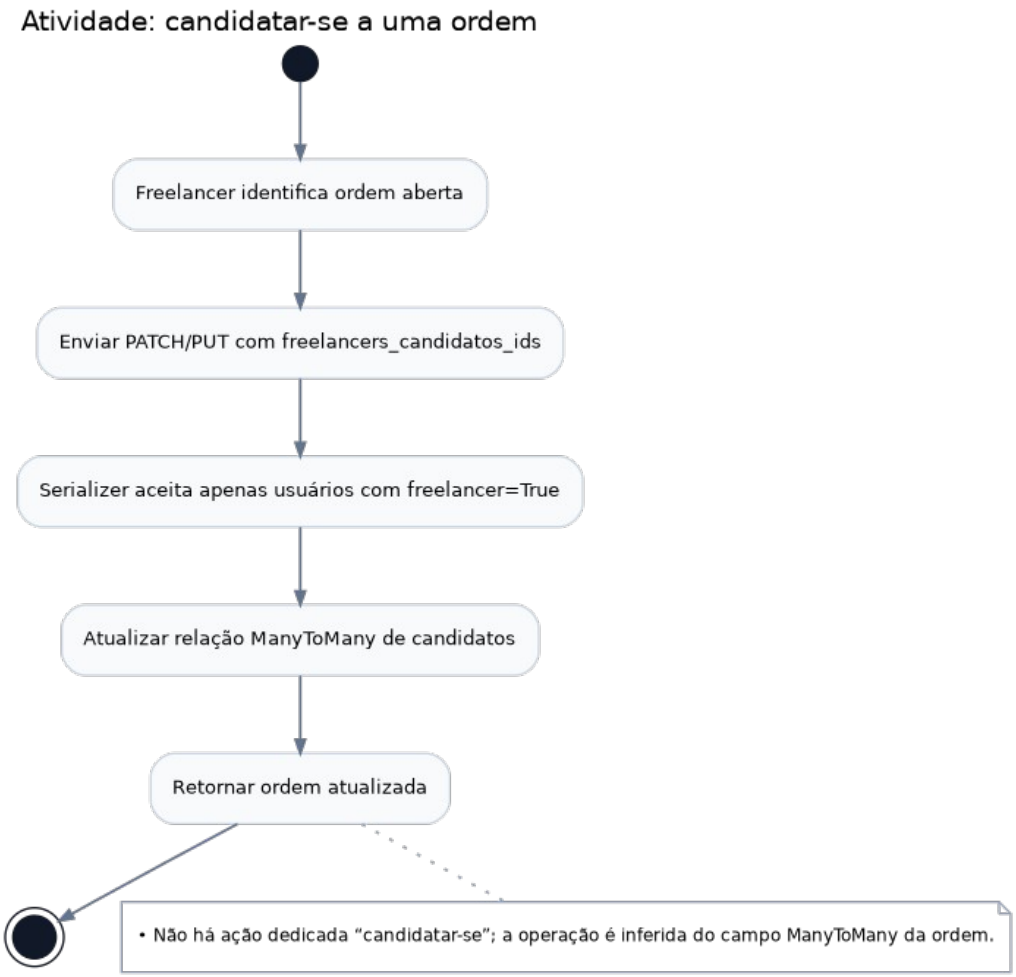

Figura 12 — Atividade de candidatura à ordem.

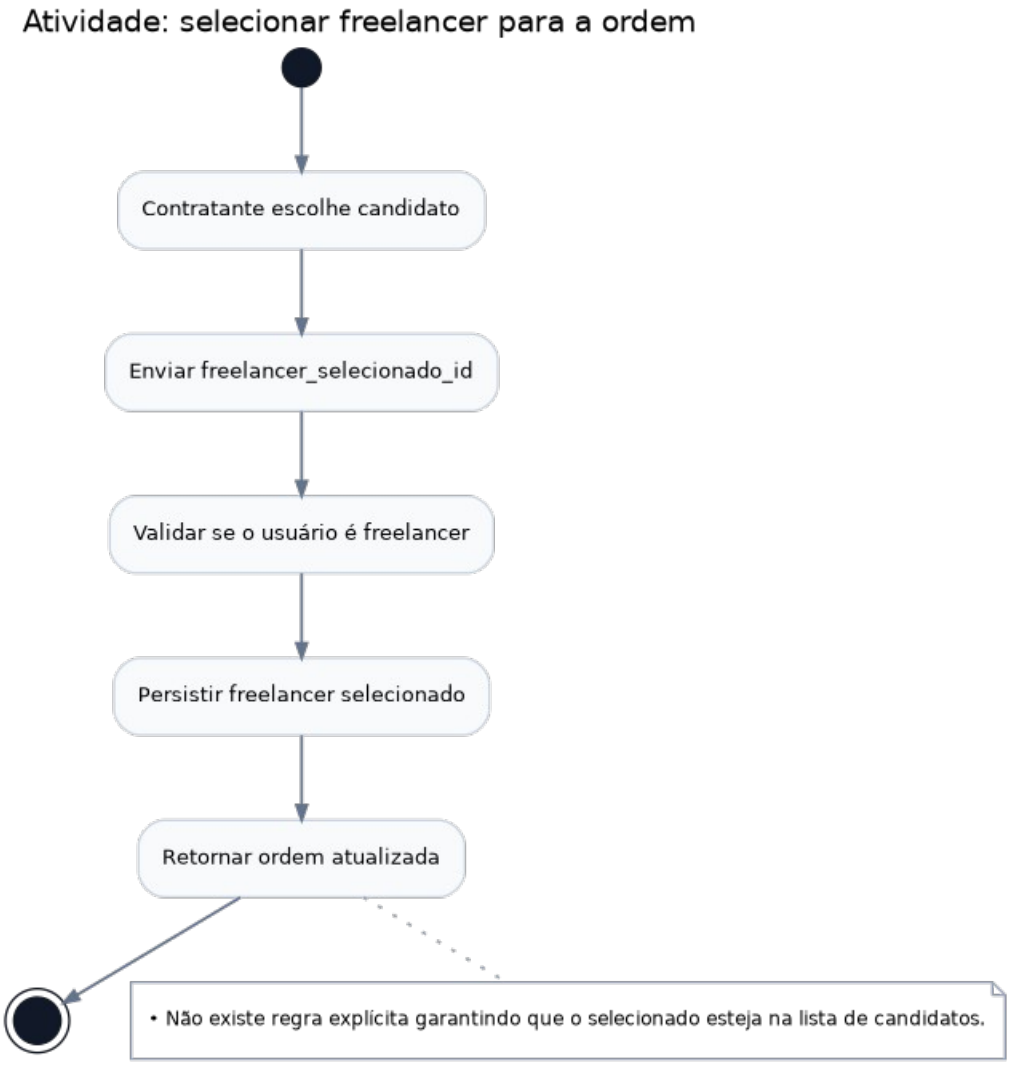

Figura 13 — Atividade de seleção de freelancer.

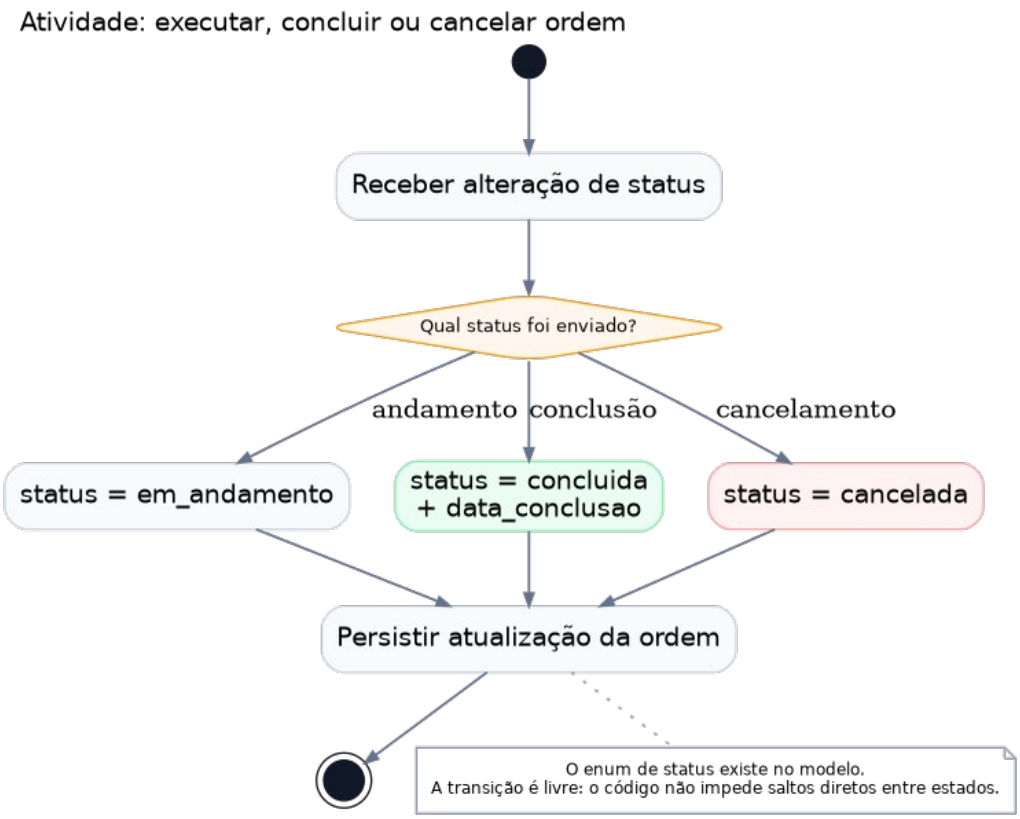

Figura 14 — Atividade de execução, conclusão ou cancelamento da ordem.

9. Diagramas de sequência

Os diagramas de sequência abaixo mostram os fluxos operacionais mais relevantes observáveis no código: cadastro de usuário, criação de ordem, candidatura/seleção e a consulta por habilidade do módulo freelancers como sequência latente.

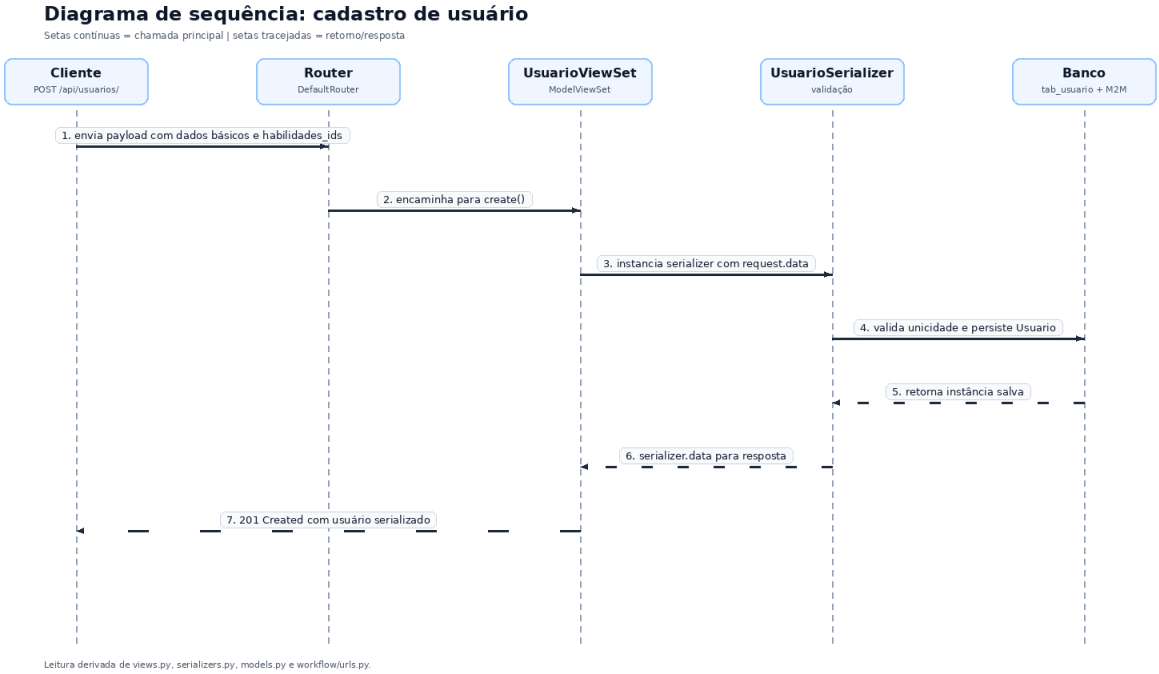

Figura 15 — Sequência de cadastro de usuário.

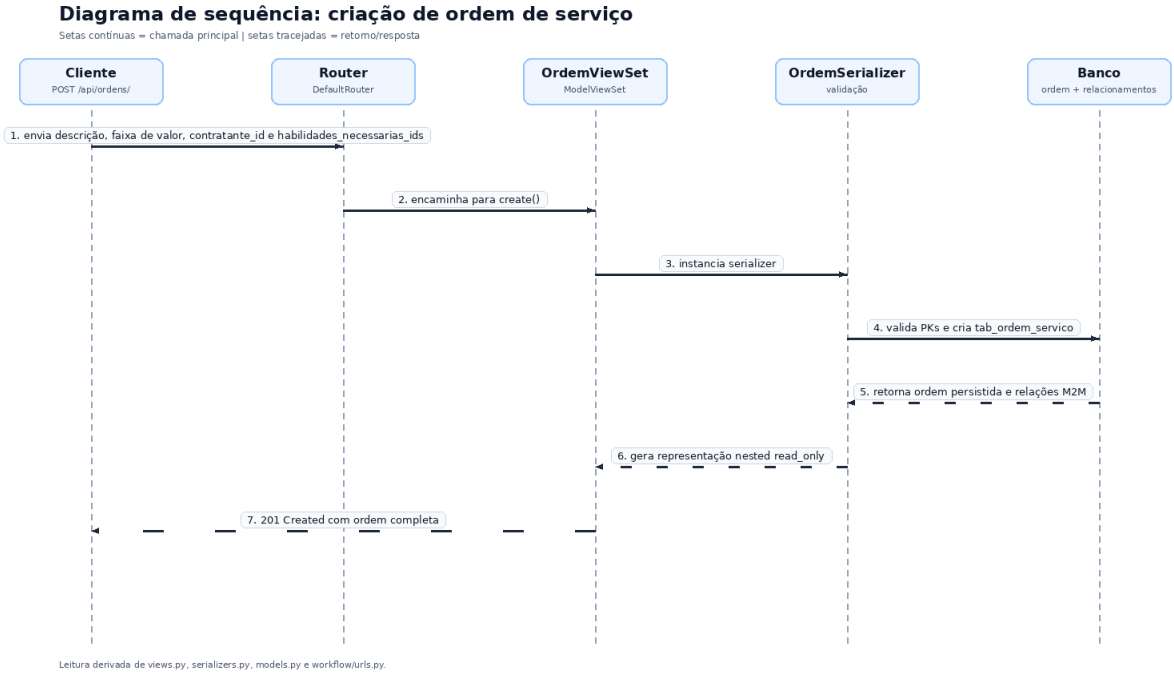

Figura 16 — Sequência de criação de ordem de serviço.

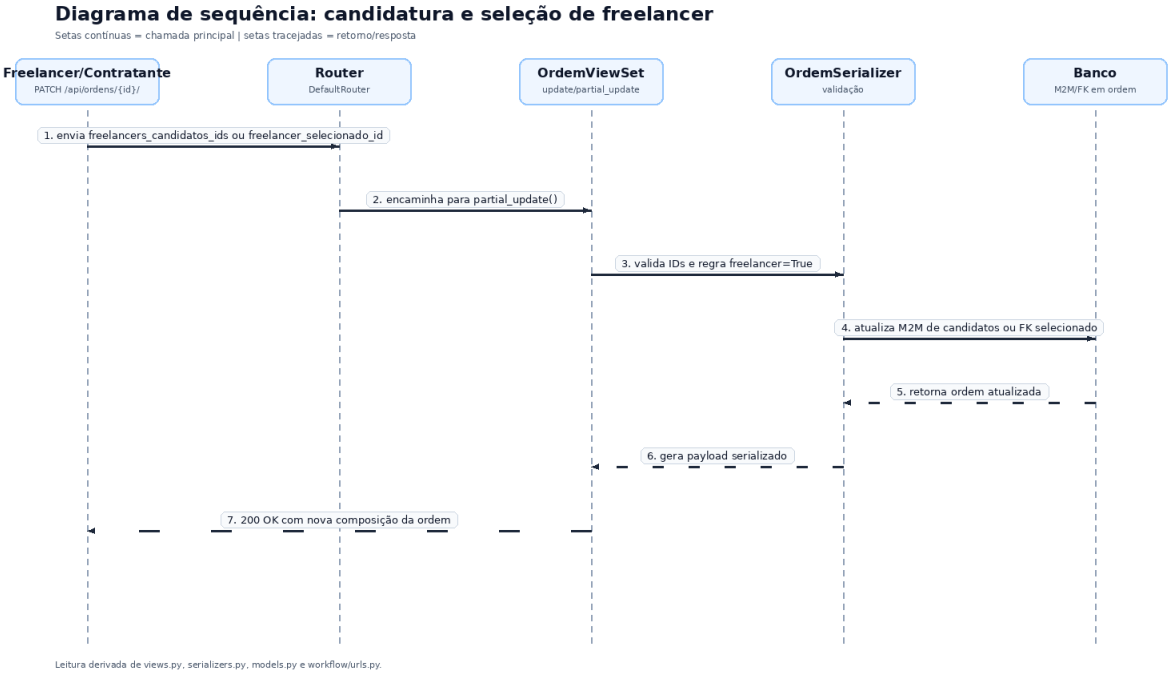

Figura 17 — Sequência de candidatura e seleção.

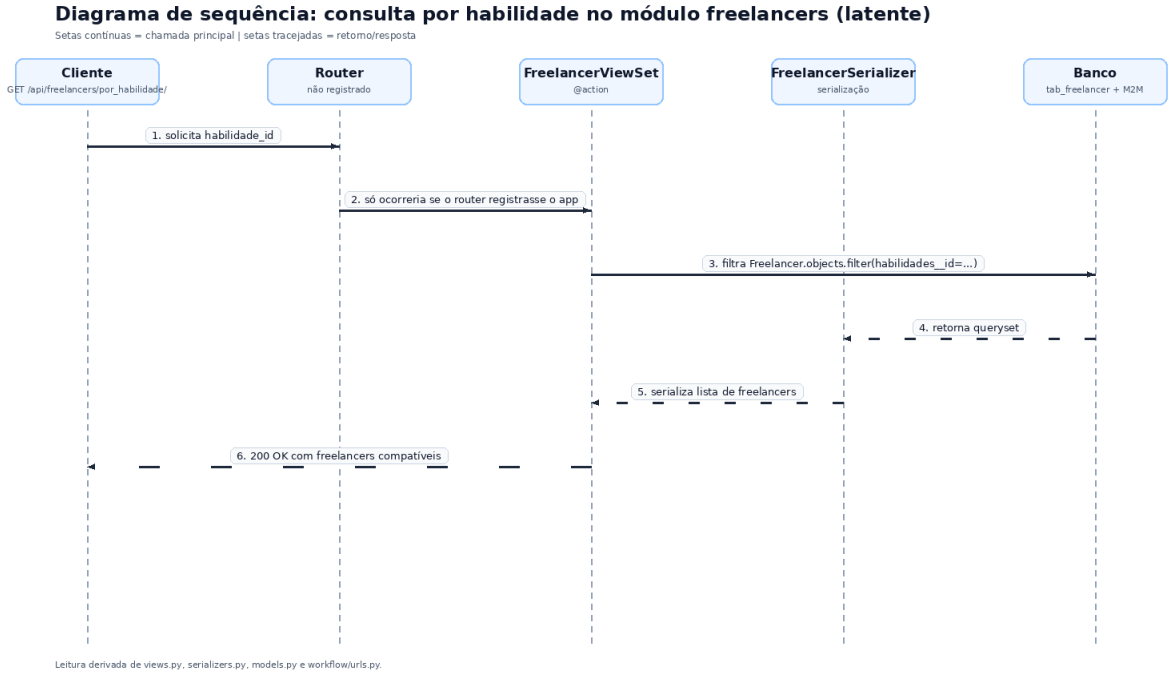

Figura 18 — Sequência latente para filtro por habilidade no módulo freelancers.

10. Achados técnicos relevantes

1. Divergência entre README e implementação — O README anuncia um endpoint /api/freelancers/, porém o router registra apenas usuarios, habilidades e ordens.

2. App freelancers ausente da configuração ativa — workflow/settings.py não inclui o app freelancers em INSTALLED_APPS, o que impede migrações, modelos e viewsets desse módulo de participarem do sistema ativo.

3. Modelagem duplicada de Habilidade — Existe Habilidade em usuarios.models e em freelancers.models apontando para a mesma tabela tab_habilidade; isso tende a gerar ambiguidade e dívida técnica.

4. Papel freelancer duplicado por duas estratégias — No fluxo ativo, freelancer é um booleano em Usuario. No fluxo legado, existe uma entidade Freelancer separada. As duas estratégias competem entre si.

5. Senha tratada de forma insegura — Usuario.senha é um CharField simples e o serializer expõe esse campo. Não há uso de hash ou integração com django.contrib.auth para o domínio de usuários do projeto.

6. Regras de negócio ainda frágeis — Não há validação explícita para garantir, por exemplo, que o freelancer selecionado pertença aos candidatos da ordem ou que a ordem siga uma máquina de estados coerente.

7. Otimização de consulta pode ser melhorada — Em ordens/views.py, prefetch_related é usado inclusive para ForeignKeys; select_related seria mais adequado para contratante e freelancer_selecionado.

8. Campos opcionais no modelo, mas potencialmente exigidos no serializer — Listas ManyToMany como habilidades_ids, freelancers_candidatos_ids e habilidades_necessarias_ids não receberam required=False, criando possível atrito de uso na API.

9. Segurança de ambiente de desenvolvimento — DEBUG=True, ALLOWED_HOSTS=["*"], CORS_ALLOW_ALL_ORIGINS=True e SessionAuthentication indicam um ambiente ainda de desenvolvimento/prototipagem.

10. Casos de uso mais maduros do projeto — Cadastro de usuários, cadastro de habilidades, criação/listagem de ordens e atualização genérica de registros já têm base funcional razoável para evoluir.

11. Conclusão

O projeto possui uma base limpa e compreensível para uma API REST simples de intermediação de serviços, com boa separação entre models, serializers e viewsets. O centro funcional está bem definido em torno de Usuario, Habilidade e OrdemDeServico. Ao mesmo tempo, a análise mostra uma bifurcação arquitetural: o domínio ativo usa Usuario com booleano freelancer, enquanto o repositório preserva um módulo freelancers alternativo que não foi efetivamente integrado. Esse ponto é o principal fator de ambiguidade estrutural.

Em termos de documentação UML, foi possível extrair diagramas de classes consolidados e por módulo, diagramas de atividades para os casos de uso principais e diagramas de sequência para os fluxos mais relevantes. Assim, a entrega cobre tanto a estrutura atual quanto as atividades de negócio possíveis a partir do código existente e do arquivo de casos de uso do próprio projeto.